Understanding executable file signing

In Windows, a file's digital signature may be stored in either a catalog signature, an embedded signature, or both. Catalog Signatures are files that contain signatures for one or more other files, while embedded signatures are stored in file-format-specific structures that allow embedding a signature within the bytes of the file itself. For example, the Windows Portable Executable (PE) File Format allows embedding a Certificate Table in the Data directory section; the signature data contains signed hashes of the code components of the file.

Signature calculations

The specific bytes protected by a file's signature vary based upon the Subject Interface Package (SIP) used to generate and later verify a signature for a given file type. Windows' signing infrastructure is extensible, allowing third-parties to register additional SIPs to support signing of additional file types beyond the signable formats included by Windows.

In the case of file types for which no SIP is installed, the entire content of the file may be hashed (as in the case of a text file) and stored in a Catalog file.

For filetypes that are supported by a SIP, like Windows Portable Executable (PE) files, the SIP may exclude some sections of the file from the hash calculation, and may store the resulting signature within the file itself or within a separate Catalog file. In a Windows PE file, digital signatures can be "embedded" in a location specified by the Certificate Table entry in Optional Header Data Directories. When Authenticode is used to sign a Windows PE file, the algorithm that calculates the file's hash value excludes certain PE fields. When embedding the signature in the file, the signing process can modify these fields without affecting the file's hash value.

After the to-be-signed content's hash is generated, it is signed using a signing certificate, allowing future validation that the hash is unmodified and that the hashed portion of the file's content remains unchanged since signing.

Security implications

The Software Publisher Trust Provider SIP for PE files hashes the executable content of the file, but necessarily omits the file's checksum (which changes when a signature is embedded) and the Certificate Table directory (which is populated by the file's signatures during signing).

The fact that the Software Publisher Trust Provider SIP does not hash all of the bytes of the file (i.e. it's not a "flat file hash") when calculating the signature means that it is possible to embed additional content into a signed PE file without breaking its signature. For example, some services will embed metadata into already-signed software installers, either by storing the metadata in an PKCS#7 padding area in the WIN_CERTIFICATE structure, by adding unvalidated attributes to a signature, or by adding an extra signature signed with a self-signed certificate. Each of these techniques introduces risk. While the injected content is not directly executable, a vulnerable application might incorrectly expect it to be trustworthy (because the rest of the file is correctly signed) and act upon it.

This unsigned-embedding capability was exploited in attacks dating back to 2013, leading to the introduction of the EnableCertPaddingCheck registry override introduced for CVE-2013-3900. When the EnableCertPaddingCheck DWORD is present and set to 1, WinVerifyTrust will validate that all PKCS#7 padding bytes in the WIN_CERTIFICATE structure are set to 0. If other values are found, the SIP will deem the signature invalid.

Notes on catalog signatures

Catalog signature verification does not support every SIP. When verifying a Catalog-signed file, the SIP is loaded only for signing and validating the PE and CAB file types. Other file types are signed and validated using a flat file hash.

Enabling the EnableCertPaddingCheck registry key does not impact WinVerifyTrust's evaluation of a Catalog-signed PE file. If an executable file is Catalog-signed, the embedded signature is not checked and thus the certificate padding is not evaluated for unexpected content even when the EnableCertPaddingCheck registry key is set.

Related content

Caveats for Authenticode Code Signing

Subject Interface Package（SIP）主题接口包是 Windows Authenticode/WinTrust 框架中的一个可插拔适配层，用来“认识”不同类型的被签名对象（即“主题”，如 PE 可执行文件、CAB 包、目录文件 .cat 等），并提供统一的接口让系统对这些对象进行签名和验签。
具体来说，SIP 以 DLL 形式实现一组规定的入口函数（常见为 SipGetSignedDataMsg、SipPutSignedDataMsg、SipCreateIndirectData、SipVerifyIndirectData、SipIsSigned、SipRemoveSignedDataMsg 等），并通过注册表把某类文件的“主题 GUID”关联到对应的 SIP。
WinVerifyTrust/WinTrust 在验证或写入签名时，会根据对象类型选择对应的 SIP：
1. 负责从对象中提取或写入 PKCS#7 等格式的签名数据（嵌入式或外置）。
2. 负责生成和验证“间接数据（Indirect Data）”，即如何对该对象计算哈希（哪些部分参与哈希、哪些字段需规避），确保签名覆盖的内容正确。
SIP 不负责“是否信任”的策略决定（那是策略提供程序 Policy Provider 的职责），它解决的是“怎么从该种对象拿到/放入签名，以及怎么正确计算用于签名的哈希”。
典型场景：
验证或添加对 PE（.exe/.dll）、驱动、CAB、MSI 等的 Authenticode 签名。
通过计算文件哈希，在目录文件（Catalog .cat）中进行匹配与验证。
第三方可以编写自定义 SIP，以支持自有封装格式的签名与验证流程。
概括来说，SIP 是 Windows 信任与签名体系中的“对象适配器”，让系统能用统一的签名/验签流程处理多种不同类型的文件或数据。

Last updated on 07/31/2025